

TOPIC 44/60

# Document Ingestion in RAG

The ETL of the AI Era.

How we transform messy, unstructured enterprise data into clean, searchable, mathematical memory.

**Ashish Jangra**

[linkedin.com/in/ashish-jangra](https://linkedin.com/in/ashish-jangra)





---

## THE CHALLENGE

# The Unstructured Chaos

Vector Databases only understand numbers (embeddings). But human knowledge is trapped in PDFs, Confluence pages, Word docs, raw HTML, and Slack messages.

### The "GIGO" Principle (Garbage In, Garbage Out)

If you feed a RAG system poorly extracted text containing broken tables, misaligned headers, or unreadable OCR, the AI will fail spectacularly—regardless of how smart the LLM is.

**Ashish Jangra**

[linkedin.com/in/ashish-jangra](https://www.linkedin.com/in/ashish-jangra)





---

## THE BIG PICTURE

# The 5-Step Ingestion Pipeline

1

**Parsing:** Extracting raw text from various file formats.

2

**Cleaning:** Removing boilerplate, noise, and formatting errors.

3

**Chunking:** Slicing text into contextually sound pieces.

4

**Embedding:** Converting chunks into mathematical vectors.

5

**Indexing:** Storing vectors + metadata into the database.

**Ashish Jangra**

[linkedin.com/in/ashish-jangra](https://www.linkedin.com/in/ashish-jangra)





## STEP 1

# Parsing & Extraction

The hardest technical challenge in RAG. A PDF is not a string of text; it's a set of visual drawing instructions for a printer.



### Text Extractors

PyPDF, pdfplumber. Good for simple documents, but fails entirely on complex multi-column layouts or tables.



### Vision & OCR

Tools like Unstructured.io or LlamaParse use AI vision models to look at the document and intelligently extract tables and charts as Markdown.

**Ashish Jangra**

[linkedin.com/in/ashish-jangra](https://www.linkedin.com/in/ashish-jangra)





STEP 2

# Cleaning & Preprocessing

Raw extracted text is incredibly noisy. If we embed the noise, the LLM will hallucinate.

RAW EXTRACTION (NOISE)

CONFIDENTIAL PAGE 42  
The company grew by 14% \n \n \n \n  
Copyright 2026. All rights reserved.  
Q3 Report

CLEANED DATA

The company grew by 14%. Q3 Report.

\* Removed headers, footers, consecutive whitespace, and boilerplate legalese.





### STEP 3

# Chunking (Recap)

We cannot embed a whole book into one vector. We slice the cleaned text into semantically relevant pieces.

A

## Determine Size

Typically 256 to 1000 tokens depending on the embedding model's capacity and the complexity of the data.

B

## Apply Strategy

Use Recursive Character, Markdown structural splits, or advanced Semantic boundary detection.

C

## Add Overlap

Always overlap chunks by 10-20% to prevent "context shearing" (losing the meaning between cuts).

**Ashish Jangra**

[linkedin.com/in/ashish-jangra](https://www.linkedin.com/in/ashish-jangra)





#### STEP 4

# Vector Embedding

This is where the magic happens. We pass every single text chunk through an Embedding Model (like OpenAI's text-embedding-3-small).

Input Chunk:

"To reset the password, click the blue link."

Output Vector (e.g., 1536 Dimensions):

[0.0124, -0.0531, 0.9932, -0.1102, 0.0431, -0.9991, 0.3341, -0.4412, ... 1528 more floats]

These coordinates mathematically represent the *concept* of resetting a password.

**Ashish Jangra**

[linkedin.com/in/ashish-jangra](https://www.linkedin.com/in/ashish-jangra)





## STEP 5

# Metadata & Indexing

Vectors alone are not enough. We must attach **Metadata** to the chunk before saving it to the Vector DB to enable *Hybrid Search*.

```
// Payload saved to DB
{
  "id": "chunk_994",
  "vector": [0.012...],
  "text": "Policy active...",
  "metadata": {
    "source": "HR_v2.pdf",
    "date": "2026-07-16",
    "department": "Sales"
  }
}
```

## Why Metadata matters?

If a user asks: *"What is the new remote work policy for Sales?"*

The DB will first filter by department == "Sales" and date > 2025, THEN perform the vector search. This guarantees precision.

**Ashish Jangra**

[linkedin.com/in/ashish-jangra](https://www.linkedin.com/in/ashish-jangra)





---

## THE GOLDEN RULE

# The 80/20 Rule of RAG

80% of your time will be spent  
building the Ingestion Pipeline.

Only 20% is spent tuning the LLM.

**Continuous Sync:** In production, Ingestion is not a one-time script. It is an automated, event-driven pipeline (like Airflow or Dagster) that constantly monitors folders, updates vectors, and deletes obsolete chunks when original files change.

**Ashish Jangra**

[linkedin.com/in/ashish-jangra](https://linkedin.com/in/ashish-jangra)





# Document Ingestion

CONCEPT UNLOCKED

- 👁️ Parsing is hard; use vision models for messy PDFs
- 🧹 Clean out boilerplate to prevent hallucination
- 🧩 Chunk properly to preserve semantic boundaries
- 🏷️ Always attach metadata to unlock Hybrid Search

CONNECT WITH ME



LinkedIn



GitHub



Kaggle



Instagram

**Ashish Jangra**

[linkedin.com/in/ashish-jangra](https://linkedin.com/in/ashish-jangra)

